

Rediscovery Summary — Updated

Big Bad Monolith — Printable Rediscovery Summary

Purpose: concise one-page summary of business rules, data model, integrations, and open questions (Liberty deployment authoritative).

1) Key Business Rules

- Users: `email` and `name` required; `email` UNIQUE (DB). See schema & DAO.
- Customers: `name` and `email` required; DAO prevents empty values.
- Billing Categories: `name` and `hourly\_rate` required; `hourly\_rate` is DECIMAL(10,2).
- Billable Hours: must reference existing Customer, User, Category; `hours` > 0; `date\_logged` required and not future; weekend logging is allowed but flagged.
- Revenue: line amount = `hours \* hourly\_rate` (service uses BigDecimal; JSP/SQL also calculates line totals).

2) Data Model (entities & important DB constraints)

- User(id, email[NOT NULL UNIQUE], name)
- Customer(id, name[NOT NULL], email[NOT NULL], address, created_at[NOT NULL])
- BillingCategory(id, name, description, hourly_rate[DECIMAL(10,2) NOT NULL])
- BillableHour(id, customer_id -> customers.id, user_id -> users.id, category_id -> billing_categories.id, hours[DECIMAL(8,2) NOT NULL], date_logged[DATE NOT NULL], created_at[TIMESTAMP NOT NULL])

3) Integrations & Config

- Liberty DataSource (production): JNDI `jdbc/DefaultDataSource` — configured in `src/main/liberty/config/server.xml`.
- Embedded Derby fallback: `jdbc:derby:./data/bigbadmonolith;create=true` — used in dev and some JSPs.
- Derby jars expected under `\${server.config.dir}/derby` per `server.xml`; `derby.database.path` is set in `bootstrap.properties`.
- Context root `/big-bad-monolith`; default ports `9080`/`9443`; dev basic user `user1`/`password` in `server.xml`.

4) Where to Verify (quick pointers)

- Schema DDL (embedded & Liberty):
`src/main/java/com/sourcegraph/demo/bigbadmonolith/dao/ConnectionManager.java` and
`../LibertyConnectionManager.java`.
- Business validation logic: `src/main/java/com/sourcegraph/demo/bigbadmonolith/service/BillingService.java` (method `validateBillableHour`).
- Sample data insertion: `src/main/java/com/sourcegraph/demo/bigbadmonolith/service/DataInitializationService.java`.
- JSPs with direct DB access and reporting SQL: `src/main/webapp/reports.jsp`, `hours.jsp`, `customers.jsp`.

5) Open Questions (need business answers)

- Deletion policy for Customers/Users/Categories with existing hours: prevent/cascade/soft-delete?
- Duplicate `users.email` UX: friendly message or current exception behavior?
- Timezone: use UTC or server-local for timestamps and reports?
- Transactions: require multi-DAO ACID transactions or per-DAO operations acceptable?
- Reporting authority: `BillingService` BigDecimal calculations or JSP/SQL outputs?
- Runtime mapping of `\${server.output.dir}` / `\${server.config.dir}` in Liberty deployment (to locate DB files).

End of summary. For a PDF export, I can generate one locally and add it to the repo if you want.

Substitution Audit (selected rows)

	trade_off
and business logic mixed in JSP scriptlets — hard to test and maintain.	Large migration effort; UI redesign and routing changes
management and duplicates connection logic; security and maintainability risk.	Moderate refactor to move SQL into DAOs/services; in
unsuitable for multi-instance/cloud deployments and scaling.	Increased ops cost for managed RDBMS but gains av
unsafe/noisy in production startup; risks seeding non-prod data.	Keep for dev/test only; ensure production disables it; r
a.time` is standard in modern Java and better supported.	Refactor effort to convert types and persistence mapp

	trade_off
Logging hinders observability; structured logging recommended.	Integrate SLF4J/logback; update logging config and de
Not safe for managed/cloud deployments; use secrets management or external identity.	Requires secrets management and potential auth flow
Difficult for repeatable migrations; use a migration tool (Flyway/Liquibase).	One-time migration to adopt tooling and remove inline
Rounding/precision risk in billing calculations.	Refactor reporting to use BigDecimal end-to-end; sma
Unclear transaction boundaries which can lead to partial updates.	Introduce transaction manager (JTA/Spring) and adjus
Assumptions are cloud-hostile (ephemeral instances, containers).	Move to platform-managed storage or managed DB; r
Scattered business logic causes divergence and maintenance burden; centralize in services.	Refactor to centralize business logic; reduces duplicat
Complex deployment to server layout; managed DBs remove the need to ship DB jars.	Adjust CI/CD and deployment artifacts; cleaner produc

Generated from rediscovery-summary.md and substitution-audit.csv